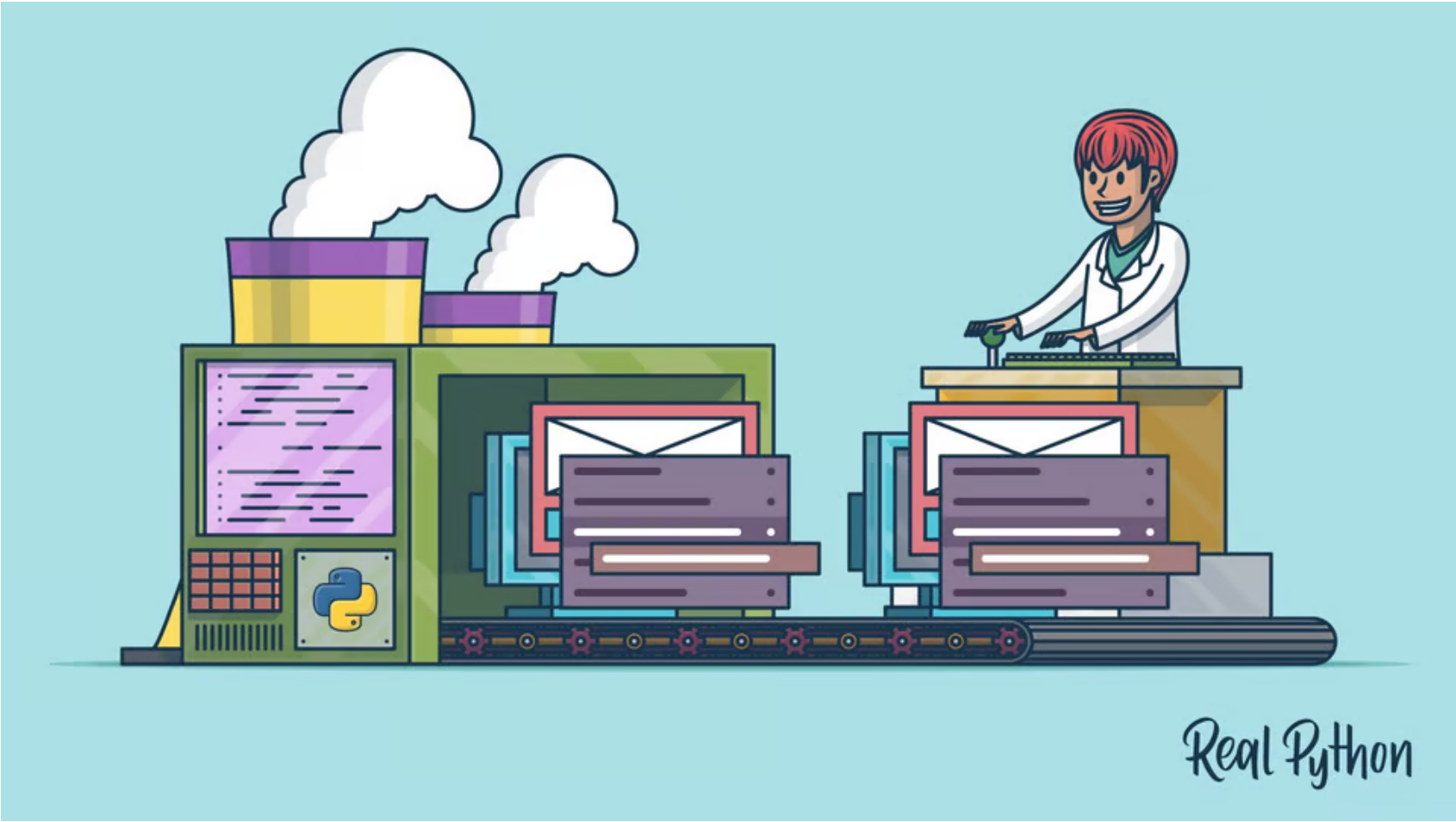
Implementing the Factory Method Pattern in Python



In this course, you'll learn how to:

- Implement the **components** of the Factory Method pattern in Python
- Improve the **design** of existing code by applying the Factory Method pattern
- Select an appropriate **implementation** of the Factory Method pattern

Table of Contents

1. Introduction of the Factory Method Pattern
2. Basic Implementation of the Factory Method Pattern
3. Advanced Implementation of the Factory Method Pattern

Table of Contents

1. **Introduction of the Factory Method Pattern**
2. Basic Implementation of the Factory Method Pattern
3. Advanced Implementation of the Factory Method Pattern

Introduction of the Factory Method Pattern

Factory Method pattern is a *creational design pattern* used to create *concrete implementations* of a *common interface*

Introduction of the Factory Method Pattern

Factory Method pattern is a **creational design pattern** used to create *concrete implementations* of a *common interface*

- Design pattern related to the *creation of objects*

Introduction of the Factory Method Pattern

Factory Method pattern is a *creational design pattern* used to create *concrete implementations* of a **common interface**

- Interface can be interpreted as the way you *interact* with an object
- For example, the public attributes and methods of a class if your object is a class
- See serializer example

Introduction of the Factory Method Pattern

Key Feature

Factory Method *separates* the process of *creating* an object from the code that depends on the *interface* of the object

- *User* of the interface (client) shouldn't worry about how the objects they are using are being *created*

Introduction of the Factory Method Pattern

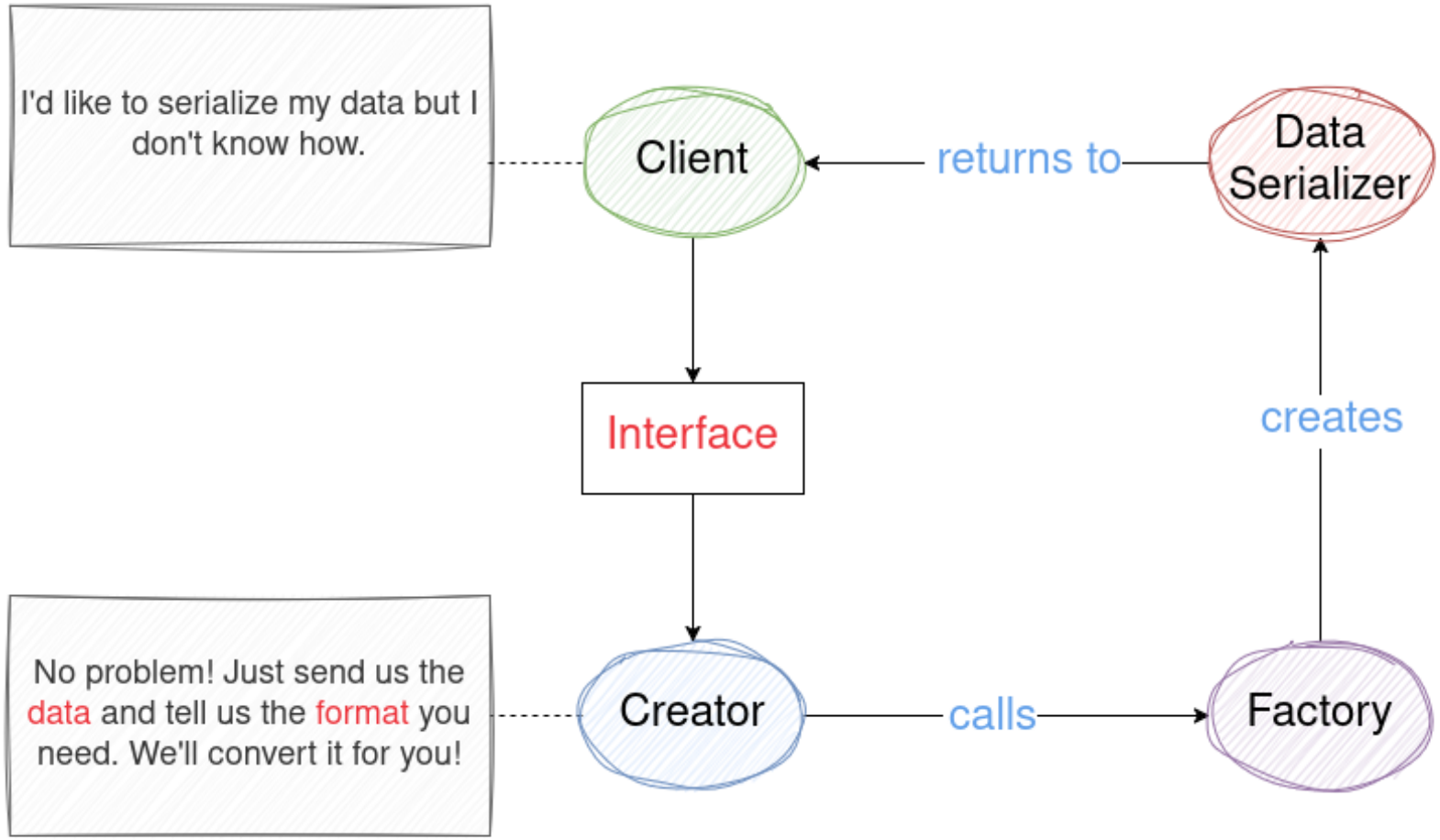
Introduction of the Factory Method Pattern

Example

Client has a library of songs and wants to convert those songs to a more convenient format such as XML or JSON

Note: the process of converting an object's state (its data and attributes) into a format that can be stored or transmitted, often as a sequence of bytes or characters, is called **serialization**

Example


I'd like to serialize my data but I don't know how.
Client
returns to
Data Serializer
Interface
creates
No problem! Just send us the data and tell us the format you need. We'll convert it for you!
Creator
calls
Factory

Example

Key Feature

Factory Method *separates* the process of *creating* an object from the code that depends on the *interface* of the object

- *Client* works with the *creator* using the *interface*
- *Creator* then takes care of creating the *products* via the *factory*

Advantages of the Factory Method Pattern

Factory Method *separates* the process of *creating* an object from the code that depends on the *interface* of the object

- **Single Responsibility Principle**: *user* of the interface (the client) shouldn't worry about how the objects she/he is using are being *created*
- **Decoupling**: reduces dependencies and improves code stability
- **Maintainability**: makes the code more modular and easier to understand and modify

Advantages of the Factory Method Pattern

Implementation of the Factory Method pattern is usually based on Object-Oriented Programming principles

- ✅ Enhanced code organization
- ✅ Reusability
- ✅ Scalability
- ✅ Maintainability

Table of Contents

1. Introduction of the Factory Method Pattern
2. **Basic Implementation of the Factory Method Pattern**
3. Advanced Implementation of the Factory Method Pattern

Install PyYAML

```
$ python -m pip install PyYAML
```

Challenges With Complex Conditional Code

- Hard to read
- Hard to understand
- Hard to maintain: `serialize()` function needs updating in too many situations

Challenges With Complex Conditional Code

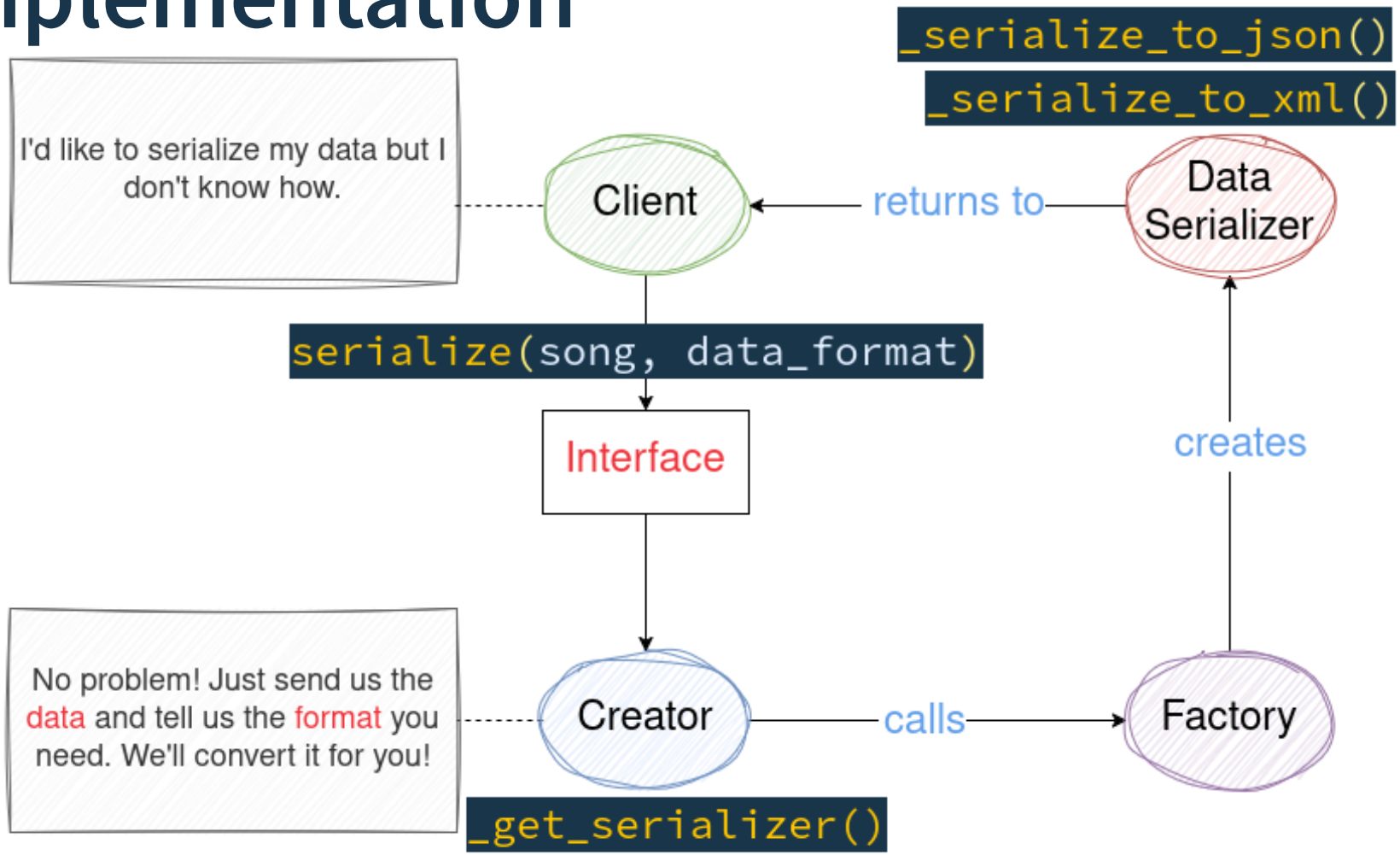
`serialize()` needs updating when:

- New format is introduced
- `Song` object changes
- Object other than `Song` needs serializing
- String representation for a format changes

Common Interface

- Common goal: convert `song` instance to string representation
- Common interface: pass `song` and `data_format` to `serialize()`

Basic Implementation



Basic Implementation of the Factory Method Pattern

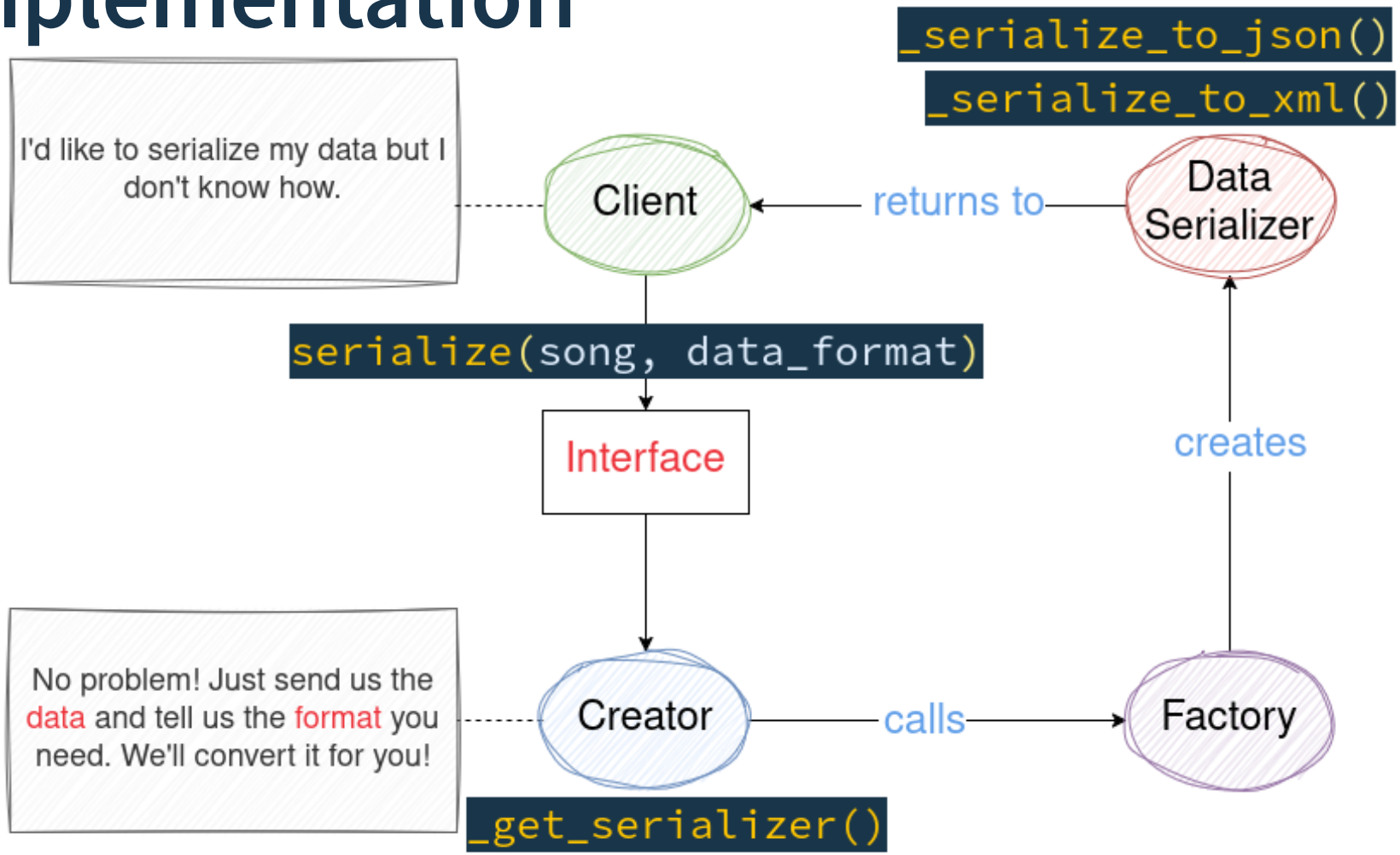
Potential improvements:

- Clearly defined factory method
- Product classes instead of functions
- Creator class instead of function
- More flexibility and structure

Table of Contents

1. Introduction of the Factory Method Pattern
2. Basic Implementation of the Factory Method Pattern
3. **Advanced Implementation of the Factory Method Pattern**

Basic Implementation



Advanced Implementation

- `.__str__()`: informal string representation of an object, aimed at the user
- `.__repr__()`: official string representation of an object, aimed at the programmer

You'll be deviating from the intented use of `.__repr__()` just to create some text output, when you're already using `__str__()`. This is just for a bit of fun and to add some lightness to the code.

Advanced Implementation

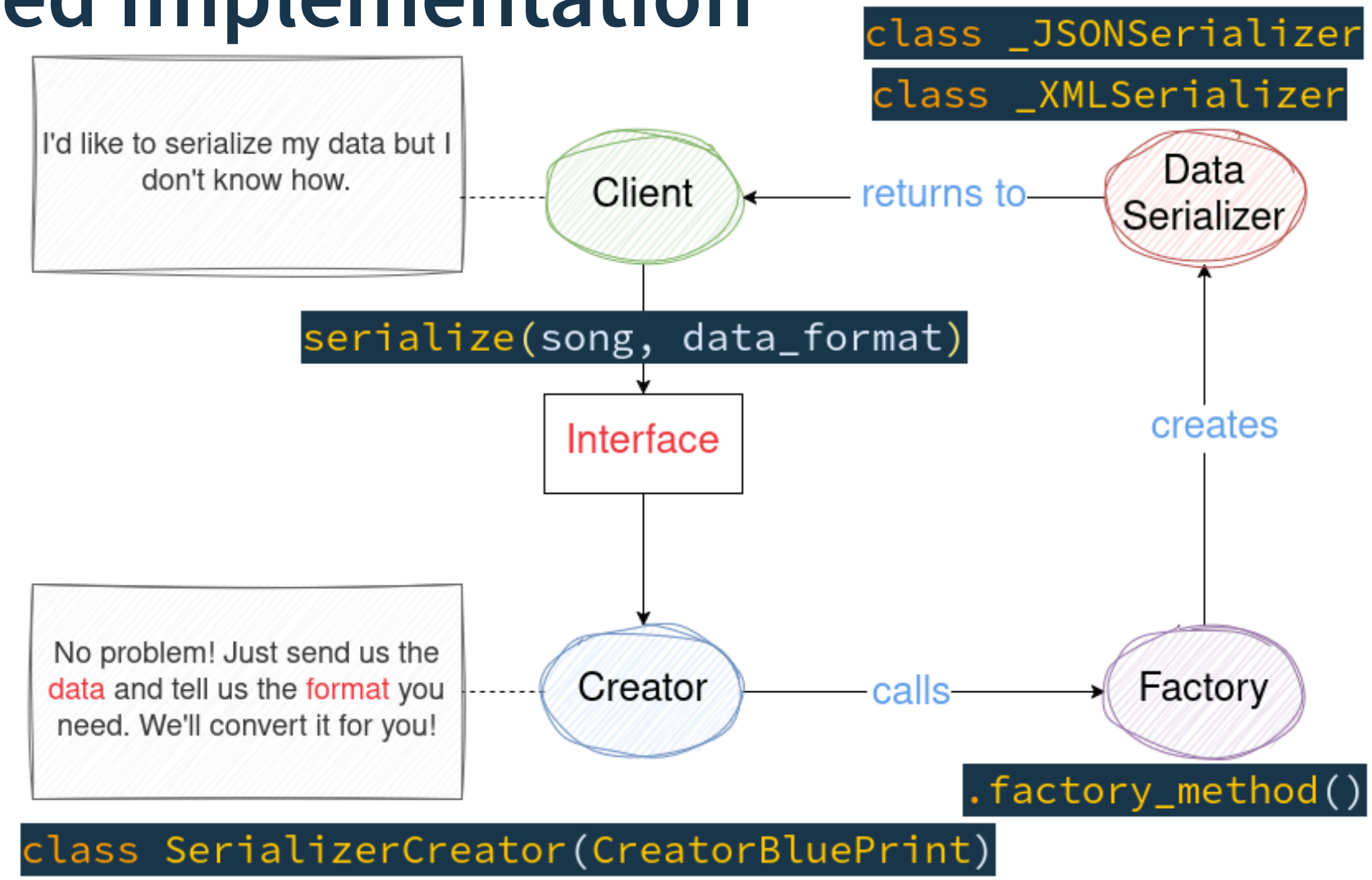
Abstract Base Classes

- Blueprints for other classes
- Define the structure that other classes must follow
- Enhance code organisation, code reusability, consistency

Install PyYAML

```
$ python -m pip install PyYAML
```

Advanced Implementation



Advanced Implementation of the Factory Method Pattern

- Product classes allow for more flexible client code
- Creator class enables increased functionality
- Abstract classes and `Protocol` bring consistency
- Product register improves ease of maintenance

Challenges with Complex Conditional Code Addressed

More understandable and readable now?

Challenges with Complex Conditional Code Addressed

More understandable and readable now?

A more complex example would prove this point more strongly

Challenges with Complex Conditional Code Addressed

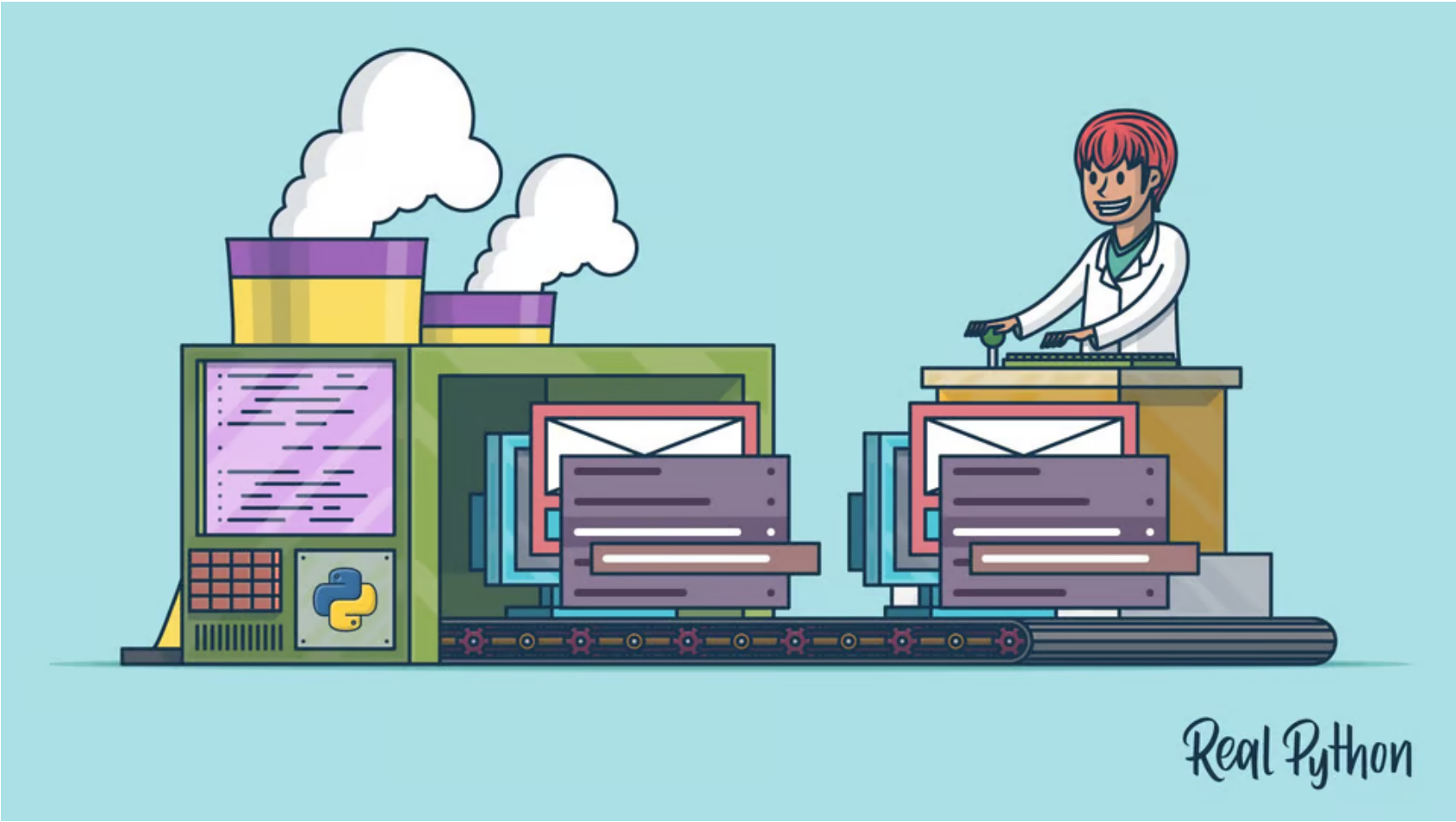
More maintainable now?

Challenges with Complex Conditional Code Addressed

More maintainable now?

- Add client objects, e.g. `Book` class
- Change existing client objects, e.g. adding an extra positional argument to the `Song` class
- Add new format added, e.g. YAML
- Let creator manage product creation

Summary



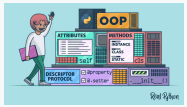
Summary

In this course, you've learned how to:

- Implement the **components** of the Factory Method pattern in Python
- Improve the **design** of existing code by applying the Factory Method pattern
- Select an appropriate **implementation** of the Factory Method pattern

Additional Resources

The Factory Method Pattern and Its Implementation in Python



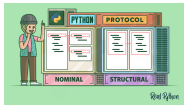
Python Classes: The Power of Object-Oriented Programming

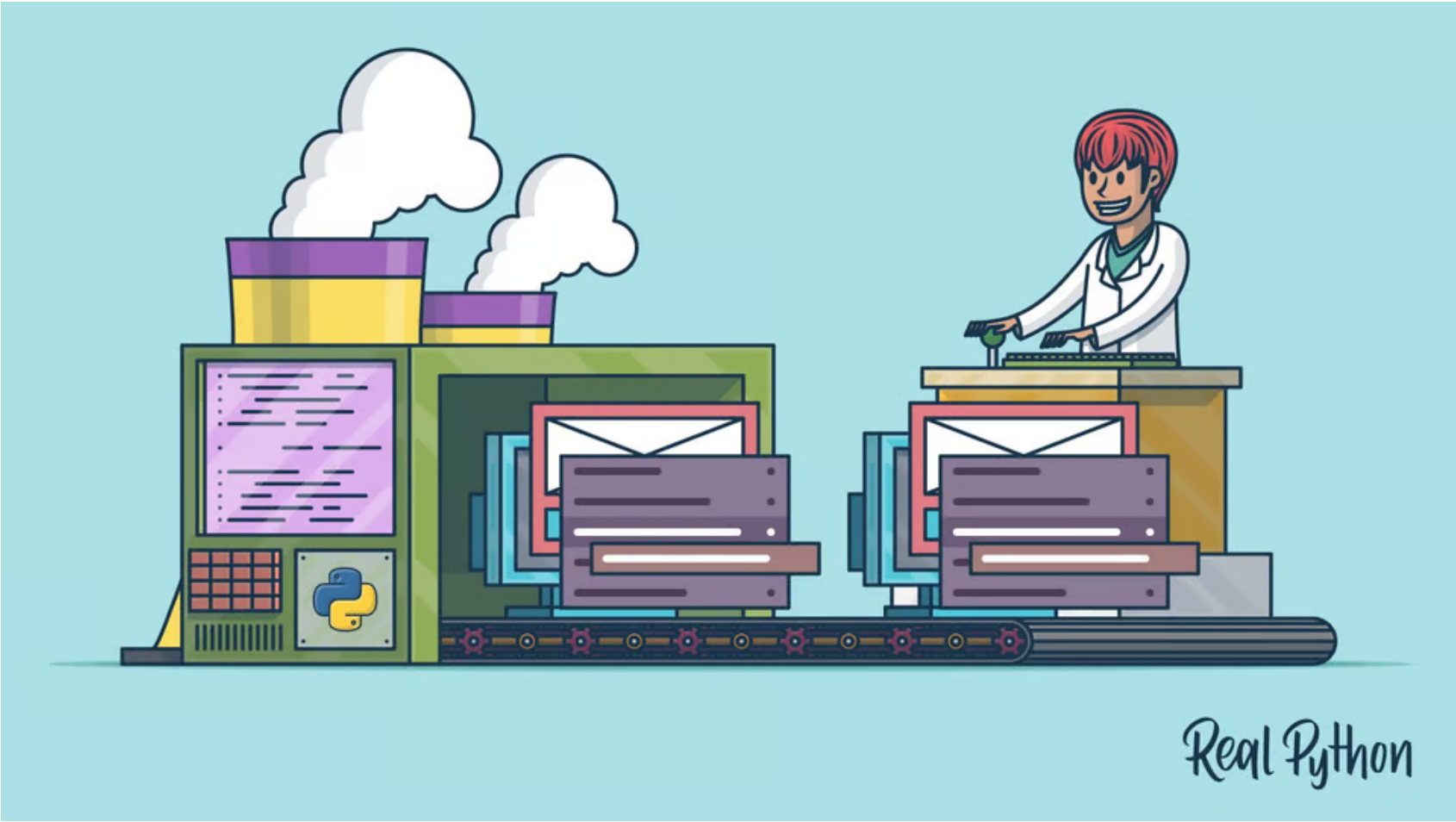
Single and Double Underscore Naming Conventions in Python

YAML: The Missing Battery in Python

Data Classes in Python 3.7+ (Guide)

abstract base class (ABC)



Python Protocols: Leveraging Structural Subtyping

Thanks for Watching!